Rapport Technique - SAÉ 4.CYBER.01

Projet : Plateforme Intelligente d'Audit de Sécurité Web - Solution automatisée de détection des vulnérabilités

Équipe : Mohammed, Emeer, Hazeem, Warid, Rayan,Fadlan

Année universitaire : 2025-2026

1. Introduction

Dans le cadre de notre formation en BUT Réseaux & Télécommunications, spécialisation cybersécurité, nous avons réalisé un projet de développement d'une plateforme de scan automatique de sécurité web. Ce projet s'inscrit dans une démarche pratique : confronter les connaissances théoriques acquises en cours à une réalisation concrète, en s'approchant autant que possible des outils et méthodes utilisés par des professionnels de la sécurité.

L'idée de départ est simple : un auditeur de sécurité doit, lors d'un audit, exécuter de nombreux outils différents, analyser leurs résultats, et en tirer un rapport. Ce processus est long, répétitif et nécessite une bonne maîtrise de chaque outil. Notre plateforme, conçue comme une solution SaaS , automatise une grande partie de ce travail. L'utilisateur n'a qu'à fournir une URL cible, et le système se charge du reste : analyse du type de cible, sélection des outils, exécution, collecte des résultats et génération d'un rapport complet.

En réponse aux attentes de notre client, ce système a été pensé pour rendre l'audit de sécurité aussi simple de compréhension qu'un audit SEO. De plus, notre approche respecte strictement le mode "White Hat" : aucune violation du système n'est réalisée et nous appliquons une temporisation pour éviter les dénis de service (DDoS) involontaires, assurant une totale bienveillance envers la cible.

L'objectif de ce rapport est de présenter de façon claire et détaillée les choix techniques que nous avons faits, le fonctionnement global du système, ainsi que les points forts, les limites et les pistes d'amélioration identifiées.

2. Conception de la plateforme

Cette section présente les décisions d'architecture et de conception qui ont guidé le développement de la plateforme. Nous détaillons ici chaque composant technique, le pourquoi de nos choix, et comment les différentes parties s'articulent entre elles.

2.1 Architecture générale

L'application suit une architecture client-serveur classique, divisée en trois grandes couches:

- **Le frontend :** Une interface web simple en HTML/CSS permettant à l'utilisateur de saisir une URL cible, de choisir le mode de scan, et de consulter les résultats. Nous avons volontairement gardé cette interface légère, car l'essentiel de la logique se passe côté serveur.

- **Le backend (Orchestrateur) :** Une API REST développée en Python avec le framework FastAPI, lancée via Uvicorn. FastAPI a été choisi pour sa rapidité de développement, sa gestion native des requêtes asynchrones et sa documentation automatique. C'est le cœur de la plateforme : il reçoit les demandes du frontend, orchestre les outils de scan, et renvoie les résultats.
- **La base de données :** Nous avons opté pour SQLite, qui est léger, ne nécessite pas de serveur dédié, et est tout à fait suffisant pour stocker les informations d'un scan. Les données sont organisées en plusieurs tables : scans, outils, résultats et vulnérabilités.

Ce découpage permet une séparation claire des responsabilités. Si demain on souhaite remplacer SQLite par PostgreSQL, ou modifier l'interface, cela ne remet pas en cause l'ensemble du projet.

2.2 Détection automatique de la cible

L'un des éléments qui distingue notre plateforme d'un simple script de scan est sa capacité à détecter automatiquement le type de cible analysée. Le backend est capable de distinguer trois types de cibles:

- **Un site web classique :** Page HTML standard, rendu côté serveur ou côté client.
- **Une API REST :** Identifiable notamment par la structure des réponses (JSON), les entêtes HTTP spécifiques et l'absence de contenu HTML traditionnel.
- **Un site WordPress :** CMS très répandu, qui possède des indicateurs caractéristiques dans le code source (chemins `/wp-content/`, `/wp-login.php`, etc.).

Une fois le type identifié, le système sélectionne automatiquement les outils les plus appropriés. Cette intelligence évite de lancer des outils inutiles et rend le scan plus efficace et plus pertinent. Il est également possible de forcer un mode manuellement.

2.3 Les outils intégrés et leur complémentarité

Nous avons intégré une large panoplie d'outils open-source reconnus. Leur complémentarité permet d'avoir une vue complète de la surface d'attaque. Voici les outils orchestrés par notre backend :

Reconnaissance et énumération :

- **Nmap :** Scanner réseau utilisé pour identifier les ports ouverts, les services qui y tournent, et la version du système d'exploitation.
- **WhatWeb :** Outil de fingerprinting web analysant les en-têtes HTTP et le code source pour identifier les technologies (CMS, serveurs, langages).
- **Dirsearch / Gobuster :** Intégrés pour l'énumération de répertoires et de fichiers cachés.
- **Amass / Subfinder :** Outils puissants pour la découverte de sous-domaines et la cartographie de la surface d'attaque externe.
- **API Probe / Headers :** Modules spécifiques pour l'analyse des en-têtes HTTP de sécurité et l'investigation des points de terminaison d'API.

Recherche de vulnérabilités et exploitation :

- **Nikto :** Scanner de vulnérabilités web générique recherchant des fichiers sensibles exposés et des configurations serveur incorrectes.
- **SQLMap :** Spécialisé dans la détection et l'exploitation des injections SQL. Nous lui avons attribué un poids important dans le calcul du score en raison de sa criticité.
- **Nuclei :** Scanner basé sur des templates (modèles de vulnérabilités) couvrant des milliers de CVE connus et des failles de configuration.
- **WPScan :** Spécialisé dans les sites WordPress pour détecter la version, les thèmes, plugins installés et leurs failles connues.
- **Dalfox :** Outil d'analyse et d'exploitation rapide focalisé sur la détection des failles XSS (Cross-Site Scripting).
- **SSLScan :** Analyse la configuration SSL/TLS du serveur cible pour vérifier la validité du certificat et détecter des problèmes cryptographiques connus.

L'association de ces outils permet de couvrir plusieurs dimensions : réseau, technologies, vulnérabilités génériques, injections, CMS, et cryptographie.

2.4 Modes de scan : normal et avancé

Pour répondre à différents besoins et respecter le cahier des charges, la plateforme propose deux modes :

- **Le mode normal (simple) :** Il exécute uniquement les outils essentiels pour la recherche des vulnérabilités les plus courantes. Le scan est rapide et permet d'avoir une première vue d'ensemble.
- **Le mode avancé :** Il active plus d'outils avec des configurations plus poussées, recherchant de manière exhaustive toutes les vulnérabilités. Le scan est plus long mais conseillé pour un audit sérieux.

2.5 Parallélisation de l'exécution

Afin d'optimiser les performances, nous avons utilisé la classe `ThreadPoolExecutor` de Python, qui permet de lancer plusieurs outils en parallèle. Pendant que Nmap scanne les ports, WhatWeb identifie les technologies et SSLScan vérifie le certificat simultanément. Les résultats sont ensuite collectés et normalisés avant de passer au scoring.

3. Conception du rapport d'audit

Un scan de sécurité n'a de valeur que si ses résultats sont présentés de façon claire et exploitable. Nous avons conçu un reporting aussi simple de compréhension qu'un rapport WooRank pour le SEO, vulgarisant les concepts techniques.

3.1 Système de scoring

Nous avons conçu un système de score sur 100 où chaque vulnérabilité fait baisser la note selon sa gravité. Pour estimer cette gravité de façon professionnelle, nous nous appuyons sur le **système de notation de l'ANSSI**, qui croise l'impact de la faille (Mineur, Important, Majeur, Critique) et sa difficulté d'exploitation (Très difficile, Difficile, Modérée, Facile).

Les niveaux de sécurité associés au score final sont :

- **90-100 : Excellent** - La cible présente peu ou pas de vulnérabilités.
- **75-89 : Good** - Quelques points à corriger mais situation globalement satisfaisante.
- **50-74 : Medium** - Plusieurs vulnérabilités présentes, des actions correctives sont nécessaires.
- **25-49 : Poor** - Situation préoccupante, des vulnérabilités sérieuses ont été détectées.
- **0-24 : Critical** - La cible est fortement exposée, des mesures urgentes s'imposent.

3.2 Contenu du rapport

Le rapport est structuré pour être lisible aussi bien par un technicien que par un responsable moins technique. Il inclut :

- **Informations générales** (URL, mode, type de cible) et **Score final**.
- **Résumé des vulnérabilités** pour une vue rapide.
- **Détail de chaque vulnérabilité** : incluant la recommandation pour y remédier. Cette vulgarisation des concepts et la fourniture d'actions correctives concrètes sont essentielles.

3.3 Formats de sortie

Le rapport est généré dans deux formats :

- **Format HTML :** Le plus pratique pour une consultation rapide à l'écran.
- **Format PDF :** Généré via ReportLab, il constitue un livrable professionnel, facilement partageable avec un client ou une hiérarchie.

4. Points forts

- **Automatisation complète et simplicité :** Du lancement jusqu'au rapport compréhensible façon "SEO", tout est automatisé.
- **Détection intelligente :** Adaptation des outils au type de cible.
- **Complémentarité massive des outils :** L'intégration de scripts sur-mesure combinant Nmap, Nikto, SQLMap, Nuclei, WPScan, Amass, Dirsearch et Gobuster offre une surface de scan exceptionnelle.
- **Rapport actionnable :** Fourniture de recommandations concrètes.
- **Architecture évolutive :** La séparation claire entre le front, l'API FastAPI et la base SQLite facilite la maintenance.

5. Limites

- **Absence d'authentification :** N'importe qui ayant accès à l'interface peut lancer un scan.
- **Risque d'abus :** Il n'y a pas de vérification que l'utilisateur est bien propriétaire du domaine ciblé.
- **Faux positifs / Négatifs :** Des outils comme Nikto peuvent générer des faux positifs, et les failles de logique métier ne sont pas détectables automatiquement.
- **Scalabilité :** SQLite et les scans parallèles simultanés peuvent limiter les performances en cas de déploiement massif multi-utilisateurs.

6. Améliorations possibles

- **Système d'authentification et de contrôle :** Ajouter un mécanisme de connexion (JWT) et exiger une preuve de propriété du domaine avant de scanner.
- **Tableau de bord interactif :** Développer une interface plus riche avec des graphiques d'évolution et la possibilité de comparer deux audits.
- **Migration vers PostgreSQL et Docker :** Conteneuriser l'application pour le déploiement et utiliser une base de données plus robuste pour la concurrence.
- **Planification :** Permettre de programmer des scans récurrents avec alertes
- (email/webhook).

7. Conclusion

Ce projet a été l'opportunité de mettre en pratique nos compétences en programmation Python, orchestration d'outils de sécurité, et vulgarisation technique. La plateforme *Web Security Scanner* répond parfaitement à l'objectif initial : fournir un système d'audit de sécurité simple d'utilisation, générant une synthèse structurée et compréhensible par des non-experts.

Bien qu'elle n'ait pas vocation à remplacer l'expertise complète d'un pentester en boîte blanche , elle constitue une solution de premier niveau fiable, rapide, et parfaitement alignée avec une démarche éthique White Hat. Ce projet nous a confirmé que la cybersécurité implique tout autant une excellence technique qu'une réflexion poussée sur l'expérience utilisateur et l'exploitabilité des résultats.